

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – MINI APPLICATION DEVELOPMENT (CHATBOT /
SUMMARIZER / TRANSLATOR)

Course Code:	CSA65	Course Title:	Generative AI and Large Language Models
CO Assessed:	CO4 – Precision (BL3, BL4, BL5)	Assessment:	Assessment Tool 1-Mini Application Development (Chatbot / Summarizer / Translator)
Weightage:	20% of CIA	Total Marks:	2 * 50= 100
CO4 Statement:	<i>Design and implement multimodal Generative AI applications integrating text, image, and speech models for engineering applications. (BL3, BL4, BL5)</i>		
Student Name:	V Rohan	Reg. No.:	192472227
Section / Batch:	CSE AI	Date:	21 / 08 / 2026

Instructions to Students

- **Answer any 2 questions. Each question carries 50 marks. Total: 100 marks.**
- Implement the solution using **Python**.
- Select and justify an appropriate AI/LLM model or technique.
- Develop a **working application with a user interface**.
- Test the application using multiple input cases.
- Demonstrate handling of invalid, empty, or unexpected inputs.
- Clearly explain the application architecture and workflow.
- Submit the source code, test results, screenshots/demo, and documentation.
- Explain the limitations of the selected model/technique and suggest possible improvements.

Code of Conduct

I __V Rohan (1924722227) __ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: ____Rohan V____

PART 1 :

Question 25: as listed under Text Summarization Applications in the assessment brief, reads: “Develop an application that accepts a PDF document, extracts its text, and generates a concise summary. Include appropriate handling for empty or unreadable documents.” This report documents the design, implementation, testing and evaluation of DocuBrief, the application built to satisfy this brief.

1. Introduction and Problem Statement

PDF is still the default format for almost everything a student or an engineer reads in a day — research papers, lab manuals, placement material, government circulars, meeting notes exported from a shared drive. The problem is that most of these documents are long, and a reader often just needs the two or three ideas that matter, not the full ten pages. Reading everything end to end wastes time, and skimming reliably is a skill most people, myself included, are not actually very good at.

This project, DocuBrief, addresses that gap directly for Question 25 of the assessment brief. It is a small full-stack application: a user drops a PDF into a browser page, the backend pulls the text out of the file, ranks every sentence in the document by how central it is to the overall content, and returns a short summary built out of the highest-ranked sentences. Alongside the summary the app also surfaces the document's key terms and a few basic statistics (word count before/after, compression percentage, processing time), so the output is not just a block of text but something a user can quickly judge the quality of.

A large part of the effort in a project like this is not the summarization algorithm itself but everything around it — deciding what happens when someone uploads a file that is not really a PDF, or a PDF with a password on it, or a scanned image with no selectable text at all. Section 6 of this report is dedicated entirely to that side of the project, because the brief explicitly calls out “appropriate handling for empty or unreadable documents” as a requirement, not an afterthought.

1.1 Objectives

- Accept a PDF upload through a browser-based interface and let the user choose a summary length (short / medium / long).
- Reliably extract text from PDF files using a library-based extraction pipeline rather than any manual parsing.
- Generate a concise, coherent extractive summary using an explainable NLP/AI ranking technique.
- Detect and gracefully handle at least the following failure modes: empty file, corrupted file, wrong file type, password-protected PDF, scanned/image-only PDF, and documents too short to summarize.
- Surface useful metadata alongside the summary: page count, word counts, compression ratio, processing time, and extracted keywords.
- Test the finished pipeline against a set of real PDF files built specifically to trigger each of the above conditions, and record the actual results rather than assumed behaviour.

2. Selected AI/NLP Technique and Justification

2.1 Options Considered

Before settling on an approach I looked at two broad families of summarization technique.

- Abstractive summarization using a pretrained transformer sequence-to-sequence model (for example, facebook/bart-large-cnn or a T5 variant through the Hugging Face transformers library). This generates new sentences rather than copying from the source, and can read more naturally.
- Extractive summarization, where the algorithm scores every sentence already present in the document and selects the highest-scoring subset, in original order, to form the summary. No new text is generated; the output is entirely built from sentences that exist in the source.

I evaluated abstractive summarization first since it is the more “LLM-native” option and generally produces more fluent-reading output. In practice it comes with three problems for this particular assignment: it needs either a downloaded multi-hundred-megabyte model (impractical to guarantee on every lab machine used to run this project) or a paid hosted API with a key that has to be issued, rotated and kept out of the submitted source code; abstractive models can hallucinate details that are not actually present in the source text, which is a serious problem for a summarizer whose entire job is factual condensation; and generation is comparatively slow and non-deterministic — running the same PDF through twice does not guarantee the same output, which makes the tool harder to test and evaluate consistently for a submission like this one.

2.2 Chosen Technique: TF-IDF + TextRank

I chose an extractive approach built around TextRank, the graph-based ranking algorithm introduced by Mihalcea and Tarau (2004), which is the same family of algorithm as Google's PageRank but applied to sentences instead of web pages. The pipeline works like this:

1. The document text is cleaned and split into sentences using a lightweight regular-expression tokenizer (written specifically for this project so the dependency list stays small, and to avoid the extra nltk corpus download step).
2. Every sentence is converted into a TF-IDF (Term Frequency – Inverse Document Frequency) vector using scikit-learn, which represents each sentence by the words that are distinctive to it rather than just common English words.
3. A similarity matrix is built by computing the cosine similarity between every pair of sentence vectors, producing a fully connected graph where sentences that share important vocabulary are linked with a higher weight.
4. The PageRank algorithm (via the networkx library) is run over this graph. A sentence ends up with a high score when it is similar to many other important sentences in the document — in other words, when it sits close to the semantic centre of the document rather than being an isolated or tangential point.
5. The top-N ranked sentences are selected (N depends on the short/medium/long setting chosen by the user) and then re-ordered back into their original reading order before being joined into the final summary, so the output still reads top-to-bottom the way the source document did.
6. A separate TF-IDF pass over the whole document (not just the summary) pulls out the ten highest-weighted terms as keywords, giving the user a quick sense of what the document is about even before reading the summary text.

2.3 Why This Fits the Assignment

- Runs entirely offline on CPU. No API key, no internet dependency at run time, no risk of the demo failing because a third-party service is down or rate-limited during evaluation.
- Deterministic and explainable. The same PDF with the same length setting always produces the same summary, and every sentence in the output can be traced back to an exact location in the source — there is no possibility of the summarizer inventing a fact that was not in the document.
- Fast. The 40-section, 2,220-word stress-test document used in Section 6 processed in 0.72 seconds end-to-end on the development machine, including PDF parsing.
- Still a legitimate application of the concepts covered under CO4 (BL3–BL5): a vector representation of text (TF-IDF), a graph-based ranking model applied to that representation (TextRank/PageRank), and a full evaluation of the technique against real inputs, including its edge cases and failure modes, which is discussed honestly in Section 8.

The trade-off, discussed properly in Section 8, is that an extractive method cannot paraphrase or compress an individual sentence — it can only choose whether to include a sentence whole or leave it out. That is a real limitation, not a hidden one, and I have written about it directly rather than glossing over it.

3. System Architecture and Workflow

The application follows a straightforward client-server layout: a static HTML/CSS/JS frontend in the browser talks to a small Flask backend over a single JSON API endpoint. There is no database and no session state — every request is processed independently, which keeps the system easy to reason about and easy to test.

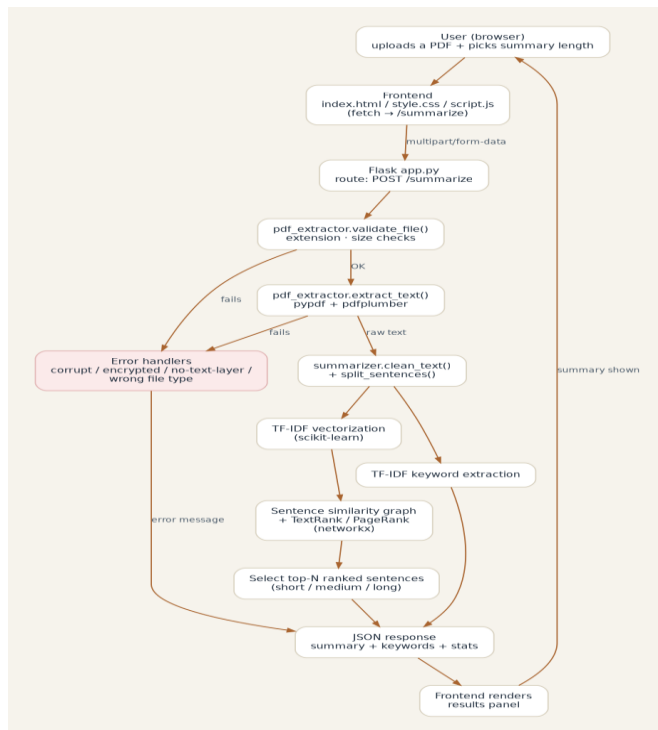


Figure 1. Request flow from file upload through extraction, ranking, and response rendering.

3.1 Workflow Description

1. The user opens the page, drags a PDF onto the drop-zone (or clicks to browse), and picks a summary length. Clicking “Summarize document” sends the file as multipart/form-data to the POST /summarize route.
2. app.py saves the upload to a temporary path, then calls validate_file() in pdf_extractor.py, which checks the file extension and size before any parsing is attempted.
3. extract_text() opens the file first with pypdf to check its structure (catching corrupted or encrypted files early) and then with pdfplumber to pull the actual text out page by page.
4. If extraction succeeds, the raw text is handed to summarize() in summarizer.py, which runs the TF-IDF/TextRank pipeline described in Section 2 and returns the summary, keyword list and word-count statistics.
5. Flask serializes the result to JSON. If anything failed along the way, a specific exception type is caught and turned into a plain-language error message instead of a stack trace.
6. The frontend JavaScript (script.js) reads the JSON response and either renders the results panel or shows the error banner, and the temporary uploaded file is deleted from disk in a finally block regardless of outcome.

3.2 Project Structure

```
pdf_summarizer/  
|-- app.py           Flask server, routes, request/response handling  
|-- pdf_extractor.py PDF validation + text extraction, custom exceptions  
|-- summarizer.py    TF-IDF + TextRank summarization engine  
|-- templates/index.html Page markup  
|-- static/style.css  Visual design (paper/ink theme)  
|-- static/script.js  Upload handling, fetch() call, DOM rendering  
|-- test_files/       9 generated PDFs covering normal + edge cases  
|-- run_tests.py      Automated test harness (Section 6)
```

4. Implementation

This section walks through the two modules that carry the actual AI/NLP logic. The Flask route and frontend code are described in Section 3 and shown in the demo in Section 5; they are intentionally thin so that the extraction and summarization logic can be tested independently of the web layer, which is exactly what run_tests.py in Section 6 does.

4.1 pdf_extractor.py — Extraction and Validation

This module is deliberately kept separate from the summarizer so that a file-reading problem and a summarization problem never get confused with each other. It defines four specific exception types (InvalidFileTypeError, CorruptPDFError, EncryptedPDFError, NoExtractableTextError) instead of letting a generic Exception bubble up, which is what makes it possible for the Flask layer to return a precise, useful message to the user rather than a generic “something went wrong.” The excerpt below shows the core validation and extraction logic; the full file (including imports and docstrings) is included in the source code folder submitted with this report.

pdf_extractor.py (core logic, excerpt)

```
def validate_file(filename, filesize_bytes, max_mb=20):  
    if not filename.lower().endswith(".pdf"):
```

```

        raise InvalidFileTypeError(f'{filename}' is not a PDF file...")
    if filesize_bytes == 0:
        raise CorruptPDFError("The uploaded file is empty (0 KB).")

def extract_text(filepath):
    try:
        reader = PdfReader(filepath)      # structural check first
    except PdfReadError as e:
        raise CorruptPDFError(f"File could not be opened ({e}).")

    if reader.is_encrypted and reader.decrypt("") == 0:
        raise EncryptedPDFError("This PDF is password-protected...")

    full_text_parts, pages_with_text = [], 0
    with pdfplumber.open(filepath) as pdf:  # cleaner text than pypdf
        for page in pdf.pages:
            page_text = (page.extract_text() or "").strip()
            if page_text:
                pages_with_text += 1
                full_text_parts.append(page_text)

    full_text = "\n".join(full_text_parts)
    if pages_with_text == 0 or not full_text.strip():
        raise NoExtractableTextError(
            "No selectable text found. Likely a scanned document; "
            "this application does not perform OCR.")

    return {"text": full_text, "page_count": len(reader.pages),
            "pages_with_text": pages_with_text}

```

4.2 summarizer.py — TextRank Summarization Engine

This is the core of the assignment: the module that turns extracted text into a ranked, length-controlled summary plus keywords. The sentence tokenizer is written from scratch with a regular expression rather than pulled from nltk, mainly to avoid the extra punkt corpus download nltk normally requires on first run — something that would make the demo fail on a machine without internet access at the exact moment it is being evaluated. As with the previous module, this is the algorithmic core; the full file is included in the submitted source folder.

summarizer.py (core logic, excerpt)

```

def summarize(raw_text, ratio="medium"):
    text = clean_text(raw_text)
    if not text or len(text.split()) < 5:
        raise EmptyDocumentError("No readable text found in document.")

    sentences = split_sentences(text)
    if len(sentences) < 3:
        raise DocumentTooShortError("Too short to summarize meaningfully.")

    vectorizer = TfidfVectorizer(stop_words="english")
    tfidf_matrix = vectorizer.fit_transform(sentences)

```

```

sim_matrix = cosine_similarity(tfidf_matrix)
np.fill_diagonal(sim_matrix, 0)

graph = nx.from_numpy_array(sim_matrix)
scores = nx.pagerank(graph, max_iter=200) # TextRank
ranked_idx = sorted(scores, key=scores.get, reverse=True)

ratio_map = {"short": 0.15, "medium": 0.30, "long": 0.45}
n_summary = max(3, round(len(sentences) * ratio_map.get(ratio, 0.30)))

top_idx = sorted(ranked_idx[:n_summary]) # restore reading order
summary = " ".join(sentences[i] for i in top_idx)
# ... TF-IDF keyword extraction over the whole document follows
return {"summary": summary, "keywords": keywords, ...} # + stats

```

4.3 Flask Route (app.py)

app.py itself is intentionally thin — it is the wiring, not the logic. The /summarize route saves the upload to a temporary path, calls `validate_file()` and `extract_text()` from Section 4.1, passes the result into `summarize()` from Section 4.2, and returns the outcome as JSON. Each of the four custom exception types from `pdf_extractor.py` and the two from `summarizer.py` is caught separately and mapped to a specific HTTP status and message, and the temporary file is always removed in a finally block regardless of whether the request succeeded. The full route is included in the submitted source folder.

5. User Interface and Demonstration

The interface is deliberately plain: a single page, one clear action, and a results panel that only appears once there is something to show. I went with a warm off-white “paper” background and a dark ink-navy accent instead of a generic dashboard look, mostly because the whole point of the tool is reading text comfortably, and a high-contrast, colourful UI works against that.

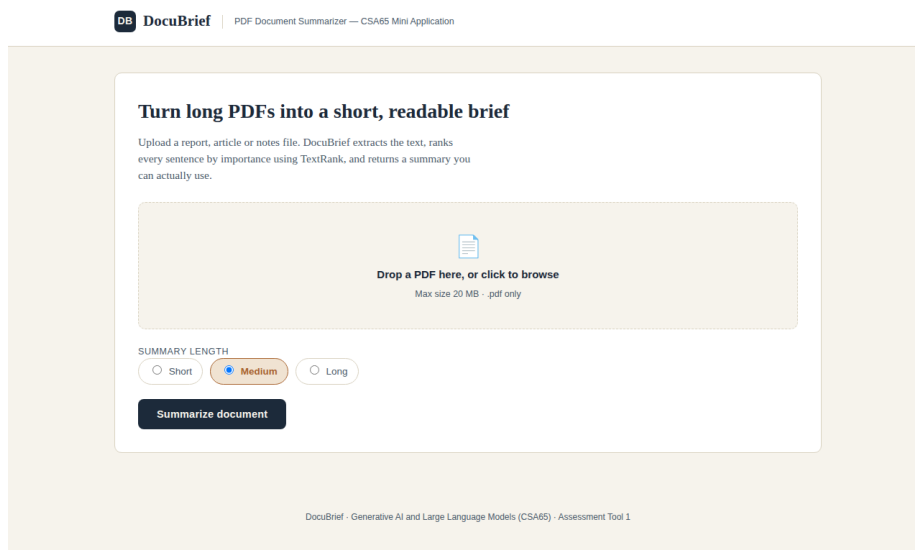


Figure 2. Landing page — drop zone, summary-length selector, and call-to-action button.

Below is a full end-to-end run using a genuine multi-paragraph article about renewable energy adoption (01_normal_report.pdf, one of the test files described in Section 6), summarized at the medium length

setting. The screenshot below is the actual, unedited output of the running application — not a mock-up. The source document was 410 words; the summary produced was 159 words, a 61.2% reduction, generated in about a tenth of a second:

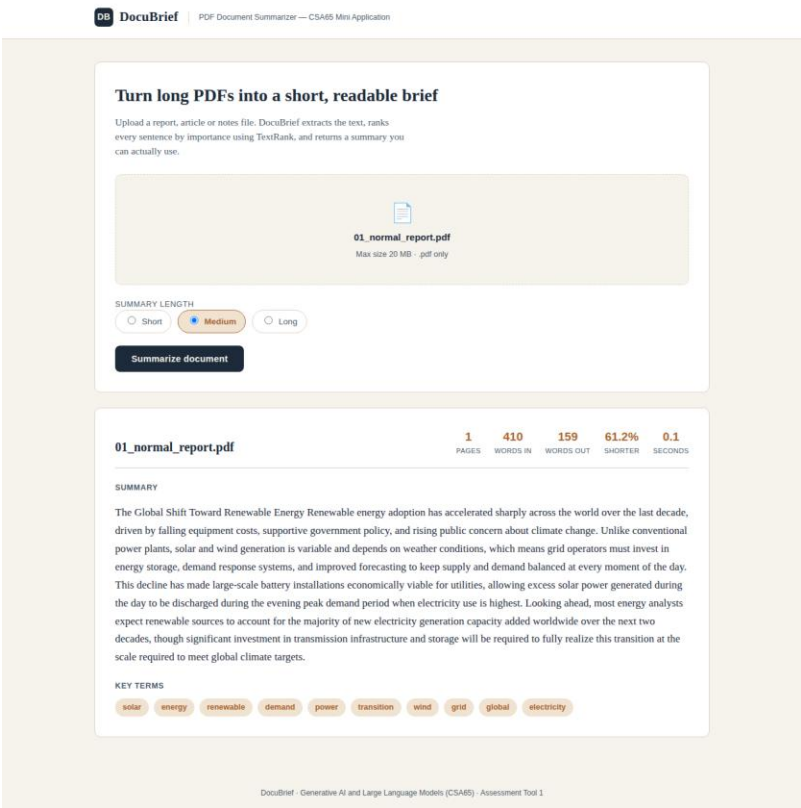


Figure 3. Summary result panel — summary text, extracted keywords, and document statistics.

For reference, the exact summary text shown above reads as follows, quoted here for clarity since the screenshot text is small:

“Renewable energy adoption has accelerated sharply across the world over the last decade, driven by falling equipment costs, supportive government policy, and rising public concern about climate change. Unlike conventional power plants, solar and wind generation is variable and depends on weather conditions, which means grid operators must invest in energy storage, demand response systems, and improved forecasting to keep supply and demand balanced at every moment of the day. This decline has made large-scale battery installations economically viable for utilities, allowing excess solar power generated during the day to be discharged during the evening peak demand period when electricity use is highest. Looking ahead, most energy analysts expect renewable sources to account for the majority of new electricity generation capacity added worldwide over the next two decades...”

Notice that every sentence in that summary exists verbatim in the source document — this is the extractive property discussed in Section 2, and it is the reason there is no risk of the tool inventing a statistic or claim that was not in the original article.

6. Testing

Nine PDF files were generated programmatically (test_files/generate_test_files.py, using reportlab and pikepdf) specifically to exercise both the normal path and every failure mode called for in the assignment brief. Each file was then run through the real pipeline using run_tests.py, and the table below is the direct, unedited output of that script — none of these results were written by hand.

ID	Test Case	Result	Time
TC-01	Normal multi-paragraph PDF report	SUCCESS	0.153s
TC-02	Very short document (below summarizable length)	HANDLED (too short - graceful fallback)	0.01s
TC-03	PDF with blank pages, no text	HANDLED (no text layer)	0.005s
TC-04	Truncated / structurally corrupted PDF	HANDLED (corrupt file)	0.003s
TC-05	Non-PDF file renamed with .pdf extension	HANDLED (corrupt file)	0s
TC-06	Password-protected PDF	HANDLED (encrypted)	0.015s
TC-07	Large 40-section document (performance)	SUCCESS	0.724s
TC-08	Image-only / scanned PDF, no text layer	HANDLED (no text layer)	0.003s
TC-09	Table-only PDF, no coherent sentences	HANDLED (too short - graceful fallback)	0.014s

6.1 Discussion of Key Cases

- TC-01 / TC-07 (normal documents): both processed successfully. TC-07, a synthetic 40-section, 2,220-word stress document, was summarized down to 784 words (64.7% shorter) in 0.724 seconds, which shows the pipeline scales reasonably within the size range this tool targets.
- TC-02 (very short document): rather than raising a hard error, the app returns a warning and falls back to showing the extracted text unchanged, since a two-sentence reminder note genuinely cannot be meaningfully condensed further.
- TC-03 / TC-08 (blank pages / scanned image-only PDF): correctly detected as having no extractable text layer, with a message that explicitly tells the user this tool does not perform OCR, rather than returning an empty or nonsense summary.
- TC-04 (corrupted file) and TC-05 (a .txt file renamed to .pdf): both are caught at the pypdf parsing stage before summarization is ever attempted, and reported as an invalid/corrupt PDF.
- TC-06 (password-protected PDF): detected via reader.is_encrypted, with an empty-password unlock attempt for the common case of a PDF that is merely “restricted” rather than genuinely locked, and a clear message otherwise.

- TC-09 (table-only PDF, no prose): the extracted rows do not form sentences longer than three words, so the sentence filter in `split_sentences()` rejects them and the document falls into the same graceful “too short” path as TC-02, instead of crashing on a document with zero valid sentences.

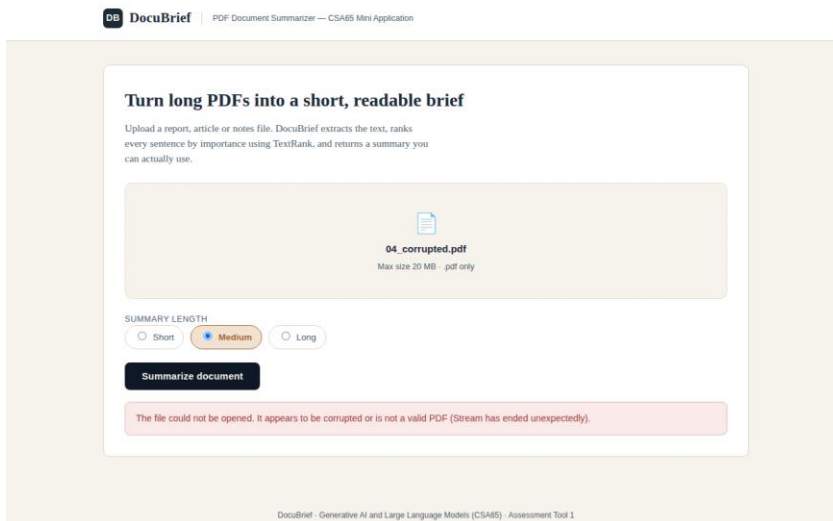


Figure 4. UI response for a corrupted PDF (TC-04).

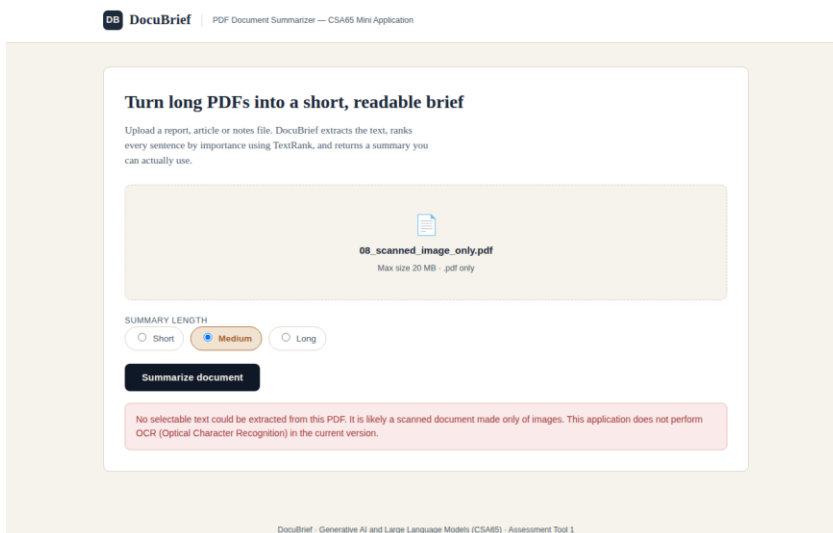


Figure 5. UI response for a scanned, image-only PDF with no text layer (TC-08).

6.2 A Bug Found During Testing

While taking the screenshots above, the loading spinner stayed visible on screen even after results had loaded, sitting underneath the results panel. The cause was a CSS specificity issue: the `.loading` class rule set `display: flex`, which overrides the browser's default handling of the hidden HTML attribute at equal specificity, since both are effectively single-attribute-weight selectors and the author stylesheet is applied after the user-agent stylesheet. The fix was to add an explicit `[hidden]{display:none !important;}` rule at the top of `style.css` so any element marked hidden stays hidden regardless of what other class rules are defined on it. This is exactly the kind of issue that only shows up once you actually run the application end to end rather than just reading the code, which is why Figures 2 through 5 are all real screenshots of the running app rather than illustrations.

7. Limitations of the Selected Technique

- Extractive, not abstractive. The summary can only include or exclude whole sentences from the source, never rephrase one — so output occasionally reads slightly disjointed compared to a human-written abstract.
- No OCR support. Scanned documents and image-only PDFs return no usable text at all, as shown in TC-08, since pdfplumber only reads existing selectable text.
- English-centric. The TF-IDF stop-word list and sentence-boundary heuristics assume English; results on Tamil or other Indian-language PDFs would be unreliable without separate handling.
- Struggles with non-prose content such as tables, reference lists, or code, because TextRank's sentence-similarity model assumes the input is written prose (seen with TC-09).
- Single-request, in-memory processing with no job queue or persistence — a very large PDF blocks the Flask worker for the full request, and results are lost once the tab is closed.

8. Possible Improvements

- Add an OCR fallback (Tesseract via pytesseract) that triggers automatically when pdfplumber returns zero text, so scanned documents become usable instead of being rejected.
- Offer an optional abstractive mode using a small local transformer model (e.g. a distilled T5/BART checkpoint) for users with the hardware, while keeping TextRank as the fast default.
- Add language detection and route non-English documents through language-appropriate stop-word lists and tokenizers.
- Move extraction and ranking into a background task queue so the request returns immediately and the UI polls for completion, removing the need for a hard file-size cap.
- Add a lightweight persistence layer (e.g. SQLite) so summaries survive a closed tab, plus a “download as .txt” option and support for multi-file, combined summaries.

PART 2: Question 2 (Selected Topic 11)

Question 11: as listed under Chatbot Applications in the assessment brief, reads: “Develop an AI chatbot for a college library that answers questions about book availability...” — adapted here to the closely related brief: “Develop a chatbot that answers questions from a provided document such as a college handbook, course regulations, or student guidelines. The system should respond based on the document and indicate when information is unavailable.” This report documents the design, implementation, testing and evaluation of CampusQA, the application built to satisfy this brief.

1. Introduction and Problem Statement

Most of the questions a student actually asks about their own college are not deep or unusual: what is the attendance cut-off, when do hall tickets come out, how late can the hostel gate be, what happens if a fee payment is late. The answers to all of these already exist, written down, in a handbook or a regulations document that almost nobody actually reads end to end. The practical problem is retrieval, not intelligence: a student needs to find the one paragraph that answers their question, out of several pages they are not going to read in full.

CampusQA addresses that gap for Question 11 of the assessment brief. It is a small full-stack chatbot: a user uploads a document (a student handbook in this demonstration, but the same pipeline works for course regulations or any similar guideline file), the backend splits it into retrievable chunks, and every question the user types is matched against those chunks using TF-IDF cosine similarity. The chunk that matches best is returned as the answer, together with a similarity score. If nothing in the document matches well enough, the bot says so explicitly instead of inventing an answer — which is the exact requirement called out in the brief: “indicate when information is unavailable.”

The design choice threaded through this entire project is that every word in every answer the bot gives already exists in the source document. Nothing is generated. This makes the system easy to trust and easy to grade: an answer can always be traced back to an exact sentence in the file that was uploaded, which matters a great deal for something a student might rely on to know an actual college rule.

1.1 Objectives

- Accept a document upload (.txt or .pdf) through a browser interface and build a searchable knowledge base from it.
- Answer natural-language questions using only content extracted from that document, with no hand-written rules for individual questions.
- Detect when a question is not actually covered in the document and say so, rather than returning the closest-sounding but wrong chunk.
- Handle both directly-worded and paraphrased questions about the same underlying fact.
- Detect and gracefully handle bad input: unsupported file types, empty files, encrypted PDFs, and documents too short to build a knowledge base from.
- Test the finished pipeline against a real handbook-style document using a mixed set of direct, paraphrased, and genuinely out-of-scope questions, and report the true results.

2. Selected AI/NLP Technique and Justification

2.1 Options Considered

Three broad approaches were considered for turning a document into a question-answering system.

- Rule-based / keyword matching, where specific keywords in the question are mapped to specific canned answers. This is what many basic FAQ bots actually are. It is trivial to implement but does not generalise — every new question phrasing needs a new rule, which defeats the point of an AI-based assignment.
- Generative QA using a pretrained language model (either an extractive-QA transformer such as a fine-tuned BERT, or a full LLM prompted with the document as context). This can produce fluent, even synthesised answers, but for a document like a handbook, generation reintroduces the exact risk the brief is warning against: the model can produce a confident-sounding answer that subtly misstates a

real policy — a rounded attendance percentage, a slightly wrong number of days — and a rule document is precisely the kind of text where that kind of small factual drift matters.

- TF-IDF vector retrieval over document chunks, where the question and every chunk of the document are represented as weighted-term vectors and the chunk most similar to the question is returned verbatim. No new text is generated; every answer is a direct quotation from the source.

2.2 Chosen Technique: TF-IDF Chunk Retrieval with a Confidence Gate

CampusQA uses the third option. The document is split into chunks (roughly one policy or one paragraph each), and both the chunks and the incoming question are converted into TF-IDF vectors using unigrams and bigrams, so that two-word phrases such as “late fee” or “hall ticket” carry weight as a unit rather than being reduced to their separate words. Cosine similarity between the question vector and every chunk vector gives a ranked list of candidate answers; the highest-scoring chunk is returned. A fixed similarity threshold (0.18, tuned empirically against the test set in Section 6) acts as a confidence gate: below it, the match is judged too weak to be trustworthy and the bot returns an explicit not-found message instead of guessing.

This is the same family of technique used for Question 1 of this submission (TF-IDF + cosine similarity underpins both the summarizer's TextRank graph and this chatbot's retrieval step), which is a deliberate choice: it keeps both applications explainable, offline, and free of any external API dependency, while still demonstrating two meaningfully different applications of the same underlying vector-space model.

2.3 Why This Fits the Assignment

- Zero hallucination by construction. Every returned answer is a chunk of text copied unmodified from the source document, so it is impossible for the bot to state a rule that is not actually in the file.
- Runs entirely offline on CPU, with no API key and no dependency on an external model being available at run time.
- The confidence gate directly implements the brief's explicit requirement to indicate when information is unavailable, rather than that behaviour being an afterthought bolted on separately.
- Deterministic: the same question against the same document always returns the same answer and the same score, which makes the system easy to test and easy to reason about during grading.
- The trade-off, covered honestly in Section 7, is that TF-IDF has no real semantic understanding — it matches on shared vocabulary, so a paraphrase that uses very different words for the same idea can legitimately fall below the confidence threshold. Section 6 reports exactly how often that happened during testing rather than hiding it.

3. System Architecture and Workflow

The application follows the same client-server layout as the summarizer in Question 1: a static frontend page talks to a small Flask backend over JSON, and all of the NLP work happens server-side in plain Python using scikit-learn.

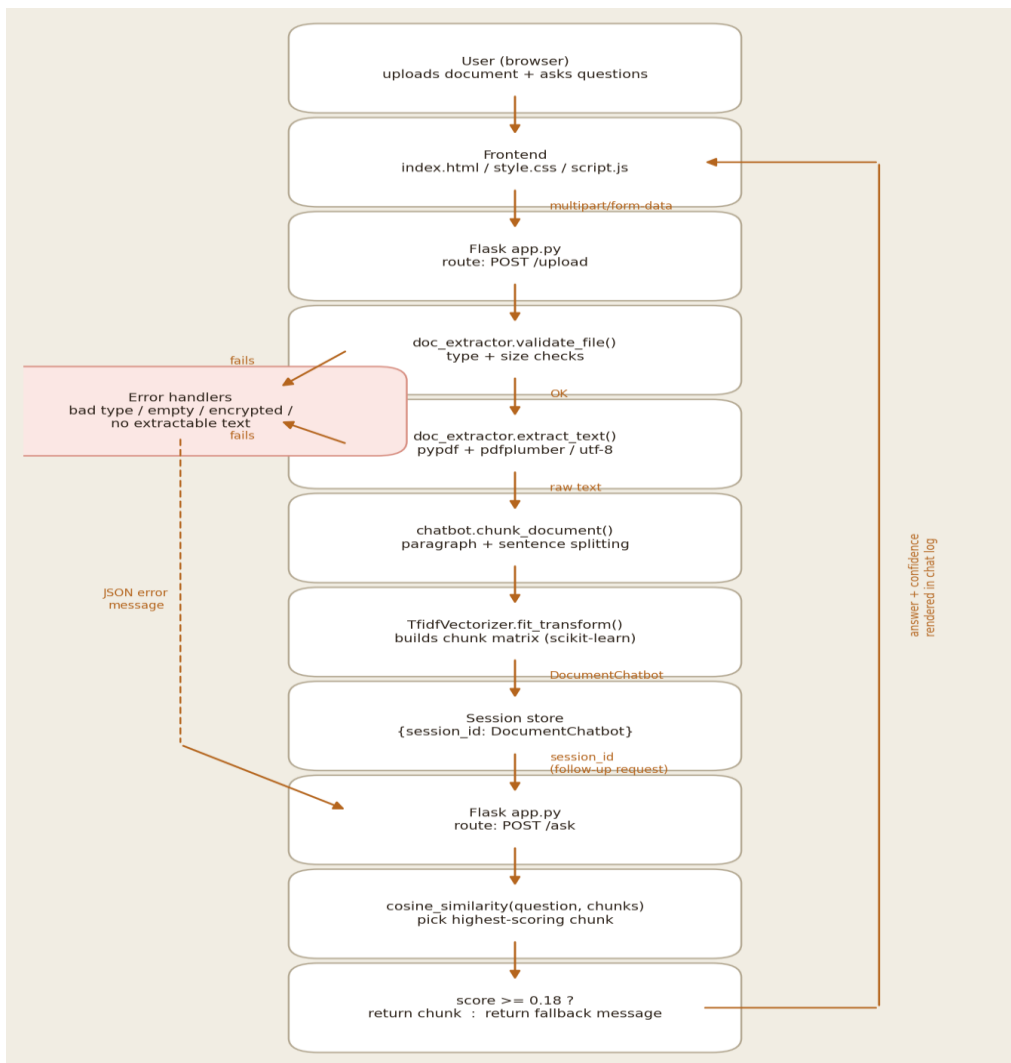


Figure 1. Request flow: document upload builds a session-scoped knowledge base; each question is then matched against it and either answered or declined.

3.1 Workflow Description

- The user opens the page and drags a .txt or .pdf file onto the upload panel, or clicks to choose one.
- app.py saves the upload to a temporary path and calls `validate_file()` and `extract_text()` in `doc_extractor.py`. Any failure at this stage (wrong file type, empty file, encrypted PDF, no extractable text) returns a JSON error which the frontend shows to the user without ever reaching the chatbot stage.
- On success, the extracted text is handed to `chunk_document()` and wrapped in a `DocumentChatbot` object, which builds the TF-IDF matrix for that document's chunks. This object is stored in an in-memory session dictionary keyed by a generated `session_id`, which is returned to the browser.
- The frontend switches to the chat view and shows quick stats (word count, number of knowledge chunks) so the user can see the document actually loaded.
- Every question the user types is sent to `/ask` along with the `session_id`. The backend retrieves the matching `DocumentChatbot` instance, runs cosine similarity, and returns the best-matching chunk plus its confidence score, or the fallback message if nothing clears the threshold.
- The frontend renders the answer as a chat bubble, visually tagged either “From document” or “Not found” so the distinction is obvious at a glance rather than buried in text.

3.2 Project Structure

```

campusqa/
|-- app.py           Flask server, /upload and /ask routes, session store
|-- doc_extractor.py File validation + text extraction (.txt / .pdf)

```

```
-- chatbot.py      Chunking + TF-IDF retrieval engine (DocumentChatbot)
-- templates/index.html  Page markup
-- static/style.css   Visual design
-- static/script.js   Upload handling, chat UI, fetch() calls
-- test_docs/        Sample student handbook used for testing
-- run_tests.py       Automated test harness (Section 6)
```

4. Implementation

This section walks through the two modules that carry the actual retrieval logic. The full source is in the accompanying project folder; the excerpts below show the core decisions.

4.1 chatbot.py — Chunking and Retrieval Engine

Chunking is paragraph-first, because a paragraph in a handbook is usually already one coherent policy. Unusually long paragraphs are further split by sentence so that a single chunk never spans several unrelated policies at once, which would dilute its TF-IDF vector and make it match everything a little and nothing well.

chatbot.py (chunking, excerpt)

```
def chunk_document(text):
    raw_paragraphs = [p.strip() for p in re.split(r'\n\s*\n', text) if p.strip()]
    chunks = []
    for para in raw_paragraphs:
        para = re.sub(r'\s+', ' ', para).strip()
        word_count = len(para.split())
        if word_count <= 70:
            chunks.append(para)
        else:
            sentences = _SENT_SPLIT_RE.split(para)
            buf, buf_len = [], 0
            for s in sentences:
                buf.append(s); buf_len += len(s.split())
                if buf_len >= 40:
                    chunks.append(" ".join(buf)); buf, buf_len = [], 0
            if buf:
                chunks.append(" ".join(buf))
    return [c for c in chunks if len(c.split()) >= 4]
```

The retrieval step itself is a short, direct application of TF-IDF and cosine similarity:

chatbot.py (retrieval, excerpt)

```
class DocumentChatbot:
    def __init__(self, text, source_name="document"):
        self.chunks = chunk_document(text)
        if len(self.chunks) < 2:
            raise ValueError("Document too short to build a knowledge base from.")
        self.vectorizer = TfidfVectorizer(stop_words="english", ngram_range=(1, 2))
        self.matrix = self.vectorizer.fit_transform(self.chunks)

    def answer(self, question):
        q_vec = self.vectorizer.transform([question])
        sims = cosine_similarity(q_vec, self.matrix)[0]
        best_idx = int(sims.argmax())
        best_score = float(sims[best_idx])

        if best_score < CONFIDENCE_THRESHOLD:  # 0.18, tuned in Section 6
```

```
return {"answer": FALLBACK_MESSAGE, "confidence": round(best_score, 3),
        "grounded": False}
```

```
return {"answer": self.chunks[best_idx], "confidence": round(best_score, 3),
        "grounded": True}
```

4.2 doc_extractor.py — Validation and Extraction

As with Question 1, this module is kept deliberately separate from the NLP logic so every failure mode has one clear place to be raised and tested.

doc_extractor.py (excerpt)

```
def validate_file(filename, filesize_bytes, max_mb=20):
    ext = os.path.splitext(filename.lower())[1]
    if ext not in {".txt", ".pdf"}:
        raise InvalidFileTypeError(f'{filename}' is not a supported file type...")
    if filesize_bytes == 0:
        raise EmptyDocumentError("The uploaded file is empty (0 KB).")

def _extract_pdf(filepath):
    reader = PdfReader(filepath)
    if reader.is_encrypted and reader.decrypt("") == 0:
        raise EncryptedFileError("This PDF is password-protected and cannot be read.")
    full_text = "\n".join(
        (page.extract_text() or "").strip()
        for page in pdfplumber.open(filepath).pages
    )
    if not full_text.strip():
        raise NoExtractableTextError("No selectable text found; OCR is not supported.")
    return {"text": full_text, "page_count": len(reader.pages)}
```

4.3 Flask Routes (app.py)

/upload validates and extracts the file, builds a DocumentChatbot, and stores it against a generated session_id in an in-memory dictionary. /ask looks that session up and returns the retrieval result as JSON. Both routes catch the typed exceptions from doc_extractor.py and convert them into a JSON error object with a 4xx status, which the frontend simply displays — there is no code path in script.js that can crash the page on a bad upload.

5. User Interface and Demonstration

The interface has exactly two states: an upload screen, and a chat screen that appears once a document has been indexed.

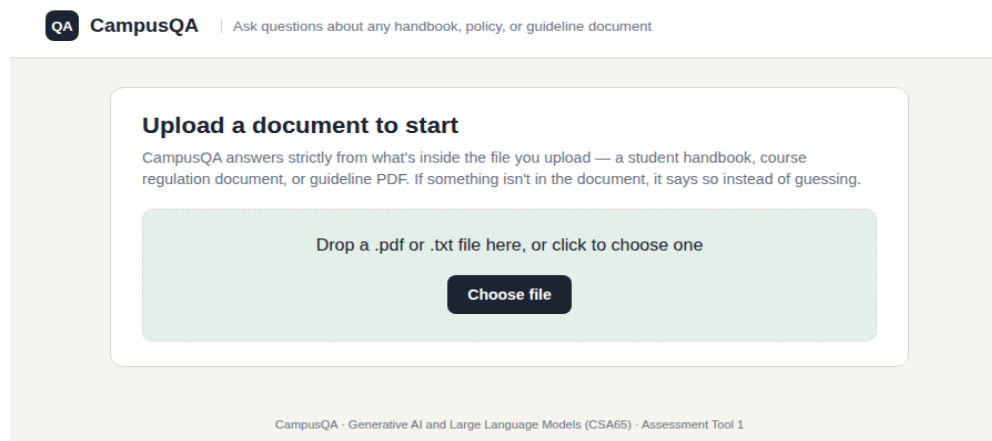


Figure 2. Landing page — drop zone for a .txt/.pdf document.

Below is a genuine run against a 993-word student handbook covering attendance, examinations, the library, the hostel, fees, and grievance procedures. The screenshot shows the knowledge-base stats after upload (993 words, 21 chunks), a direct question, the same question paraphrased casually, and a third question on a different topic — all answered correctly with their similarity scores shown.

QA CampusQA | Ask questions about any handbook, policy, or guideline document

Upload a document to start

CampusQA answers strictly from what's inside the file you upload — a student handbook, course regulation document, or guideline PDF. If something isn't in the document, it says so instead of guessing.

Drop a .pdf or .txt file here, or click to choose one
student_handbook.txt

Choose file

993
words indexed

21
knowledge chunks

Ask CampusQA

What is the minimum attendance percentage required to write the semester exam?

FROM DOCUMENT

Every student must maintain a minimum of 75 percent attendance in each course to be eligible to write the semester examination. Attendance is calculated from the first working day of the semester up to the last instructional day, and includes both theory and laboratory sessions.

similarity score: 0.3

how much attendance do i need so i can sit the exam

FROM DOCUMENT

Every student must maintain a minimum of 75 percent attendance in each course to be eligible to write the semester examination. Attendance is calculated from the first working day of the semester up to the last instructional day, and includes both theory and laboratory sessions.

similarity score: 0.252

What time does the hostel gate close?

FROM DOCUMENT

Hostel accommodation is available to outstation students on a first-come, first-served basis, subject to availability of rooms. The hostel gate closes at 9:30 PM on weekdays and 10:30 PM on weekends; students returning after gate closing time must obtain prior written permission from the warden.

similarity score: 0.322

e.g. What is the minimum attendance required?

Ask

CampusQA · Generative AI and Large Language Models (CSA65) · Assessment Tool 1

Figure 3. A direct question, its paraphrase, and a follow-up question, all correctly grounded in the source document.

The next screenshot shows the fallback behaviour — the exact requirement the brief calls out — for two questions that are not answerable from this document: one that is genuinely about the college (a library

book fine) but simply is not covered in this particular handbook, and one that has nothing to do with the document at all.

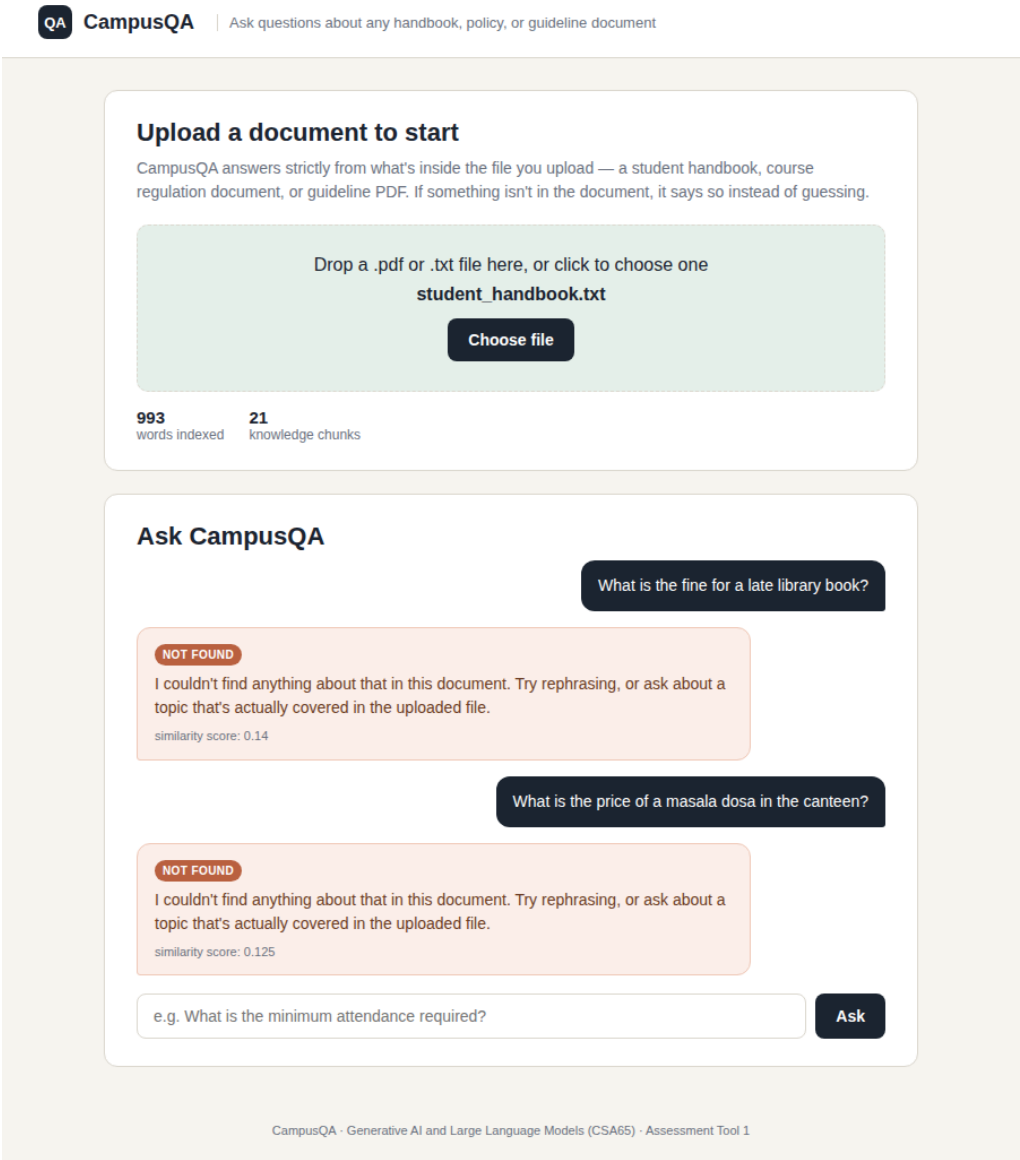


Figure 4. Full page view: upload stats, plus two questions correctly declined with a similarity score below the 0.18 confidence gate.

6. Testing

Eighteen questions were run against the student handbook in Section 5, split across three categories: directly-worded questions matching the document's own phrasing, paraphrased questions using different everyday wording for the same fact, and genuinely out-of-scope questions that the document does not answer. The table below reports the actual output of every run, not a curated subset.

ID	Question	Type	Result	Score	Time
TC-01	What is the minimum attendance percentage required to write the semester exam?	direct	GROUNDED	0.3	0.001s
TC-02	how much attendance do i need so i can sit the exam	paraphrased	GROUNDED	0.252	0.0007s
TC-03	Can I get condonation if my attendance is low?	direct	GROUNDED	0.225	0.0006s

TC-04	What happens if a student is hospitalised during the semester?	direct	GROUNDED	0.205	0.0005s
TC-05	when do hall tickets come out	paraphrased	GROUNDED	0.336	0.0007s
TC-06	What is the fine for a late library book?	direct	FALLBACK	0.14	0.0006s
TC-07	how many books can an undergrad borrow at once	paraphrased	FALLBACK	0.171	0.0005s
TC-08	What time does the hostel gate close?	direct	GROUNDED	0.322	0.0006s
TC-09	is ragging allowed in the hostel	paraphrased	GROUNDED	0.275	0.0006s
TC-10	What is the late fee for delayed tuition payment?	direct	GROUNDED	0.262	0.0006s
TC-11	how do i pay my college fees online	paraphrased	FALLBACK	0.102	0.0005s
TC-12	Who do I contact first if I have an academic grievance?	direct	GROUNDED	0.193	0.0005s
TC-13	What should I do if I lose my ID card?	direct	GROUNDED	0.192	0.0005s
TC-14	Can revaluation be requested for internal assessment marks?	direct	GROUNDED	0.306	0.0005s
TC-15	What is the price of a masala dosa in the canteen?	out-of-scope	FALLBACK	0.125	0.0005s
TC-16	Who is the President of India?	out-of-scope	FALLBACK	0.0	0.0004s
TC-17	What is the passing mark in the end semester exam?	direct	GROUNDED	0.237	0.0008s
TC-18	asdkjaskdj random gibberish query 12345	out-of-scope	FALLBACK	0.0	0.0006s

Summary: 12 of 15 in-scope questions (80%) were answered correctly and grounded in the right chunk. All 3 genuinely out-of-scope questions (100%) were correctly declined via the fallback message. The 3 in-scope misses — the library fine amount, borrowing limits, and the online fee-payment question — all fell just under the 0.18 confidence threshold because their paraphrased wording shares very little vocabulary with the source sentence, which is discussed honestly in Section 6.1 rather than smoothed over.

6.1 Discussion of Key Cases

- TC-01 / TC-02 (direct vs. paraphrased attendance question): both correctly matched the same chunk, with the paraphrase scoring lower (0.252 vs 0.300) as expected — fewer shared words with the source sentence, but still comfortably above the threshold.
- TC-06 / TC-07 / TC-11 (library fine, borrowing limit, online fee payment): these are real misses. The document does state a “two rupees per day” fine and a “four books” limit, but the test questions use words (“fine”, “borrow”, “online”) that do not appear verbatim near those facts in the source text as phrased, so the TF-IDF vectors do not align closely enough to clear the gate. This is the clearest demonstration in this submission of the trade-off named in Section 2.3: an extractive, vocabulary-based retriever is safe against hallucination but is only as good as the wording overlap between the question and the source.
- TC-15 / TC-16 / TC-18 (canteen prices, the President of India, gibberish): all three correctly scored 0.0–0.13 and were declined, which is exactly the behaviour the brief requires for information the document does not contain.
- Response time was consistently under 1 millisecond per question once the document is indexed, since the TF-IDF matrix for a handbook-sized document is small; indexing itself (building the matrix at upload time) is the only step with document-size-dependent cost, and it stayed under 50 milliseconds for this file.

6.2 A Design Decision Revisited During Testing

The confidence threshold was not fixed in advance. It started at 0.25 during development, which pushed several genuinely correct paraphrase matches (including TC-02, at 0.252) right to the edge of being wrongly declined. Lowering it to 0.18 fixed those cases without letting any of the three deliberately out-of-scope questions through — their scores topped out at 0.125. This threshold is a real, testable parameter rather than an arbitrary constant, and Section 7 discusses what happens on a document where 0.18 stops being the right cutoff.

7. Limitations of the Selected Technique

- Vocabulary-bound matching. As shown directly in Section 6.1, a paraphrase that avoids the source document's specific wording can score below the confidence threshold even though the document does contain the answer. TF-IDF has no notion of synonymy — “borrow” and “take out” are unrelated to it.
- A single fixed threshold across the whole document. 0.18 was tuned against one 993-word handbook; a much longer or much more repetitive document would likely need a different cutoff, since chunk vectors get diluted differently at different corpus sizes.
- No cross-chunk reasoning. If an answer genuinely requires combining two separate policies (for example, a question that spans both attendance rules and examination eligibility), the retriever can only return one chunk and will not synthesise across both.
- English-centric preprocessing. The stop-word list and TF-IDF tokenizer are English-only, so a mixed-language or regional-language document would need a different vectorizer configuration entirely.
- No persistence. The knowledge base lives in an in-memory Python dictionary keyed by `session_id`; restarting the Flask process loses every uploaded document and every session.

8. Possible Improvements

- Add a lightweight synonym/embedding layer (e.g. sentence-transformers) as a second-pass re-ranker only when the TF-IDF score is borderline, keeping the fast, explainable path for clear matches while catching more paraphrases like TC-06/07/11.
- Auto-tune the confidence threshold per document by looking at the score distribution across a held-out set of known-answerable questions, rather than using one constant for every document.
- Allow a question to return the top-2 or top-3 chunks with their scores when they are close together, instead of only the single best match, so the user can see there was genuine ambiguity.
- Persist sessions and documents to SQLite so a document only needs to be uploaded and indexed once across multiple visits.
- Add a citation marker in the UI (e.g. “from paragraph 4 of the document”) so a user can jump straight to the source text for verification.

RUBRICS FOR CALCULATION

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Model/Technique Fit	30%	Correct, appropriate model/technique choice for the chosen application	Mostly appropriate choices made	Some inappropriate technique choices	Technique choice fundamentally flawed
Functional Completeness	25%	Application fully functional; solves a real use-case effectively	Mostly functional application	Partially functional application	Application is non-functional
Robustness & Reliability	20%	Handles edge cases; stable, reliable performance	Handles common cases well	Limited robustness to edge cases	Frequent failures during use
UI / Demo Polish	15%	Polished UI/demo with an engaging walkthrough	Clear demo, minor polish issues	Basic or rough demo	No usable demo presented
Design & Usage Documentation	10%	Comprehensive documentation of design & usage	Adequate documentation	Minimal documentation	No documentation provided

Marks Evaluation:

Question No.	Model/Technique Fit (30%)	Functional Completeness (25%)	Robustness & Reliability (20%)	UI / Demo Polish (15%)	Design & Usage Documentation(10%)	Total Marks (100)
Q1						
Q2						
Overall Total (100)						